**UNIVERSITÉ HASSAN 1er – SETTAT**
**École Nationale des Sciences Appliquées BERRECHID**

Département de Mathématique et Informatique
**Filière : Ingénierie des Systèmes d'Information et Big Data**

# Guide d'Installation

# Apache Airflow

## Installation d'Apache Airflow avec Docker Compose

Orchestrateur de workflows de données – version 2.11 avec PostgreSQL



**Module :** Big Data et Applications

**Réalisé par :**
Salma Bouziane Ouaritini

**Encadré par :**
Pr. Lamia Karim

**Année Universitaire 2026–2027**

# Table des matières

**1 Introduction** **2**

**2 Installation et vérification de Docker Desktop** **2**
2.1 Vérification de l'état de Docker . . . . . 2

**3 Préparation de la structure du projet** **2**

**4 Configuration du fichier docker-compose.yml** **2**
4.1 Explication détaillée des paramètres de configuration . . . . . 4

**5 Lancement de l'application** **4**

**6 Vérification des conteneurs en cours d'exécution** **4**

**7 Accès à l'interface graphique Apache Airflow** **5**

**8 Test : création d'un premier DAG** **5**
8.1 Explication du code . . . . . 5
8.2 Exécution du DAG . . . . . 6

**9 Arrêt et relance d'Apache Airflow** **6**

**10 Problèmes fréquents** **6**

**11 Conclusion** **6**

# 1. Introduction

**Apache Airflow** est une plateforme open source qui permet de créer, planifier et superviser des chaînes de traitement de données (*workflows*). Chaque chaîne est décrite en Python sous la forme d'un **DAG** (*Directed Acyclic Graph*) : un ensemble de tâches reliées entre elles, exécutées dans un ordre précis. Airflow fournit une interface web pour lancer les DAG, suivre l'état de chaque tâche et consulter leurs logs.

Ce document constitue un guide d'installation pas à pas pour déployer Apache Airflow sous Windows, en utilisant la conteneurisation Docker et l'outil d'orchestration Docker Compose. Cette approche garantit un environnement de travail isolé, reproductible et rapidement opérationnel.

L'installation comporte deux conteneurs :

— **PostgreSQL**, la base de données dans laquelle Airflow enregistre l'état des DAG et des exécutions ;
— **Airflow**, qui regroupe le *scheduler* (qui déclenche les tâches) et le *webserver* (l'interface graphique).

# 2. Installation et vérification de Docker Desktop

Télécharger Docker Desktop pour Windows depuis le site officiel (https://www.docker.com/products/docker-desktop/) et procéder à l'installation en veillant à activer l'option **WSL 2** (Windows Subsystem for Linux).

## 2.1. Vérification de l'état de Docker

Lancer Docker Desktop, puis ouvrir l'Invite de commandes (CMD) ou PowerShell et exécuter la commande suivante pour vérifier la version de Docker :

```
docker --version
docker compose version
```

# 3. Préparation de la structure du projet

Créer un dossier de travail dédié sur la machine hôte, ainsi qu'un sous-dossier `dags` qui contiendra les fichiers Python des DAG :

```
mkdir airflow-projet
cd airflow-projet
mkdir dags
```

Le dossier `dags` sera partagé avec le conteneur : tout fichier ajouté dans ce dossier sera automatiquement détecté par Airflow.

# 4. Configuration du fichier docker-compose.yml

Dans le dossier `airflow-projet`, créer un fichier nommé `docker-compose.yml` et y insérer la configuration suivante :

```
services:
  postgres:
    image: postgres:16-alpine
    container_name: airflow-postgres
    environment:
```

```
      - POSTGRES_USER=airflow
      - POSTGRES_PASSWORD=airflow
      - POSTGRES_DB=airflow
    volumes:
      - postgres-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "airflow"]
      interval: 10s
      retries: 5
    restart: unless-stopped

  airflow:
    image: apache/airflow:2.11.2-python3.11
    container_name: mon-airflow
    depends_on:
      postgres:
        condition: service_healthy
    environment:
      - AIRFLOW__CORE__EXECUTOR=LocalExecutor
      - AIRFLOW__DATABASE__SQL_ALCHEMY_CONN=postgresql+psycopg2://airflow:airflow@postgres/airflow
      - AIRFLOW__CORE__LOAD_EXAMPLES=false
      - AIRFLOW__WEBSERVER__SECRET_KEY=une-cle-secrete-a-changer
    ports:
      - "8080:8080"
    volumes:
      - ./dags:/opt/airflow/dags
    command:
      - bash
      - -c
      - >-
        airflow db migrate &&
        (airflow users create --username admin --password admin
        --firstname Admin --lastname Airflow --role Admin
        --email admin@example.com || true) &&
        (airflow scheduler &) &&
        exec airflow webserver
    restart: unless-stopped

volumes:
  postgres-data:
```

### 4.1. Explication détaillée des paramètres de configuration

| Directive / Paramètre | Description & Rôle |
|---|---|
| `image: postgres:16-alpine` | Télécharge l'image officielle de PostgreSQL, base de métadonnées d'Airflow |
| `POSTGRES_USER / PASSWORD / DB` | Crée l'utilisateur, le mot de passe et la base `airflow` |
| `volumes: postgres-data` | Conserve les données de la base même si le conteneur est supprimé |
| `healthcheck` | Vérifie que PostgreSQL est prêt avant de démarrer Airflow |
| `image: apache/airflow:2.11.2-python3.11` | Télécharge l'image officielle d'Apache Airflow 2.11.2 (Python 3.11) depuis Docker Hub |
| `container_name: mon-airflow` | Attribue un nom explicite au conteneur pour simplifier sa gestion |
| `depends_on (service_healthy)` | Airflow ne démarre qu'une fois la base de données disponible |
| `AIRFLOW__CORE__EXECUTOR` | `LocalExecutor` : les tâches s'exécutent localement, plusieurs en parallèle |
| `AIRFLOW__DATABASE__SQL_ALCHEMY_CONN` | Adresse de connexion à la base PostgreSQL |
| `AIRFLOW__CORE__LOAD_EXAMPLES` | `false` : n'affiche pas les DAG d'exemple fournis avec Airflow |
| `AIRFLOW__WEBSERVER__SECRET_KEY` | Clé utilisée pour sécuriser les sessions de l'interface web |
| `ports: - "8080:8080"` | Rend l'interface web accessible sur le port 8080 de la machine hôte |
| `volumes: ./dags` | Partage le dossier local `dags` avec le conteneur |
| `command` | Initialise la base (`db migrate`), crée le compte `admin`, lance le scheduler en arrière-plan puis le serveur web |
| `restart: unless-stopped` | Redémarre automatiquement le conteneur en cas d'interruption non planifiée |

## 5. Lancement de l'application

Depuis le terminal, exécuter la commande suivante dans le répertoire du projet pour lancer les conteneurs en arrière-plan :

```
docker compose up -d
```

Au premier lancement, Docker télécharge les images d'Airflow et de PostgreSQL (environ 1,5 Go) : cette étape peut prendre plusieurs minutes selon la connexion.

## 6. Vérification des conteneurs en cours d'exécution

Pour confirmer que les conteneurs `airflow-postgres` et `mon-airflow` sont bien actifs, exécuter :

```
docker compose ps
```

Les deux conteneurs doivent apparaître avec l'état `Up`, et PostgreSQL avec la mention `healthy`. Au premier démarrage, Airflow initialise sa base de données, ce qui prend une à deux minutes. Pour suivre cette initialisation :

```
docker compose logs -f airflow
```

Attendre la ligne indiquant que le serveur web écoute sur le port 8080, puis quitter l'affichage

avec `Ctrl + C`.

## 7. Accès à l'interface graphique Apache Airflow

Ouvrir un navigateur web et saisir l'adresse suivante :

```
http://localhost:8080
```

Se connecter à l'aide des identifiants créés au démarrage :

- **Username :** `admin`
- **Password :** `admin`

La page **DAGs** s'affiche. Elle est vide pour l'instant, puisque le dossier `dags` ne contient encore aucun fichier.

## 8. Test : création d'un premier DAG

Pour vérifier que l'installation fonctionne, on crée un DAG simple composé de deux tâches. Dans le dossier `dags`, créer un fichier nommé `hello_airflow.py` et y insérer le code suivant :

```python
from datetime import datetime

from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.operators.python import PythonOperator


def dire_bonjour():
    print("Bonjour depuis Apache Airflow !")


with DAG(
    dag_id="hello_airflow",
    start_date=datetime(2024, 1, 1),
    schedule=None,
    catchup=False,
    tags=["test"],
) as dag:

    date = BashOperator(task_id="afficher_date", bash_command="date")
    bonjour = PythonOperator(task_id="dire_bonjour", python_callable=dire_bonjour)

    date >> bonjour
```

### 8.1. Explication du code

| Élément | Rôle |
|---|---|
| `dag_id="hello_airflow"` | Nom du DAG affiché dans l'interface |
| `schedule=None` | Le DAG ne se lance pas automatiquement : il est déclenché à la main |
| `catchup=False` | Airflow ne rattrape pas les exécutions passées depuis `start_date` |
| `BashOperator` | Tâche qui exécute une commande système (ici `date`) |
| `PythonOperator` | Tâche qui exécute une fonction Python (ici `dire_bonjour`) |
| `date >> bonjour` | Définit l'ordre : `afficher_date` s'exécute avant `dire_bonjour` |

### 8.2. Exécution du DAG

1. Patienter environ 30 secondes, le temps qu'Airflow détecte le nouveau fichier, puis actualiser la page **DAGs** : le DAG `hello_airflow` apparaît.
2. Activer le DAG avec l'interrupteur situé à gauche de son nom (les nouveaux DAG sont en pause par défaut).
3. Cliquer sur le nom du DAG, puis sur le bouton **Trigger DAG** (le triangle, en haut à droite).
4. Ouvrir l'onglet **Graph** : les deux tâches passent successivement au vert foncé (*success*).
5. Cliquer sur la tâche `dire_bonjour`, puis sur l'onglet **Logs** : le message `Bonjour depuis Apache Airflow !` s'affiche.

**RÉSULTAT :** si les deux tâches sont vertes et que le message apparaît dans le log, Apache Airflow est correctement installé et opérationnel.

## 9. Arrêt et relance d'Apache Airflow

| Commande | Effet |
|---|---|
| `docker compose stop` | Arrête les conteneurs en conservant leur état |
| `docker compose start` | Relance les conteneurs arrêtés |
| `docker compose down` | Supprime les conteneurs ; la base de données est conservée dans le volume |
| `docker compose down -v` | Supprime aussi le volume : l'historique et les utilisateurs d'Airflow sont effacés |

## 10. Problèmes fréquents

| Problème | Solution |
|---|---|
| `localhost:8080` ne répond pas | Airflow est encore en cours d'initialisation : attendre une à deux minutes et consulter `docker compose logs airflow` |
| Le port 8080 est déjà utilisé | Remplacer `"8080:8080"` par `"8081:8080"` et ouvrir `http://localhost:8081` |
| Le DAG n'apparaît pas | Vérifier que le fichier est bien dans le dossier `dags` ; une erreur de syntaxe est affichée en haut de la page **DAGs** |
| Le DAG apparaît mais ne s'exécute pas | Vérifier qu'il est activé (interrupteur à gauche de son nom) |
| Identifiants refusés | Le compte n'a pas encore été créé : attendre la fin de l'initialisation |
| `Docker Desktop is unable to start` | Le disque C : est plein : libérer de l'espace, exécuter `wsl -shutdown` puis relancer Docker Desktop |

## 11. Conclusion

Ce guide a permis d'installer Apache Airflow 2.11 avec Docker Compose, accompagné de sa base de métadonnées PostgreSQL, puis de vérifier son bon fonctionnement en créant et en exécutant un premier DAG. Les étapes essentielles sont :

— installer Docker Desktop avec WSL 2 ;
— créer le dossier du projet et le fichier `docker-compose.yml` ;
— lancer les conteneurs avec `docker compose up -d` ;

— se connecter à l'interface sur `http://localhost:8080` ;
— déposer un fichier Python dans le dossier `dags` pour créer un DAG, puis le déclencher depuis l'interface.

Cet environnement constitue la base sur laquelle un pipeline de données complet peut ensuite être orchestré, en ajoutant de nouveaux DAG dans le dossier `dags`.